# 第 11 章 排序

## 1. 为什么要学排序？

可能同学在学习排序算法的时候会有一个疑惑：无论是 C 还是 C++，都已经提供了 `sort` 函数，为什么还要花大量的时间去学习排序算法呢？不如把时间用在学习别的算法上。这个想法是片面的。

- 虽然接下来要学习的排序算法，在以后做题中一半以上都不会用到。但是，不要单纯的想着往后做题用到它们。这些算法思想，将会伴随我们学习后面的知识。比如堆排序中的**贪心**思想，归并排序里面的**分治**思想，快速排序里面的**递归**思想等。学习这些算法能起到预习的作用，后续学到对应算法的时候能够更加得心应手。
- 除了算法思想，我们在学习这些算法的过程中，还会锻炼：如何处理写代码时的细节问题，如何优化算法，遇见 bug 如何调试，分析时空复杂度等等。

因此，希望大家不要抱着无所谓的心态来学习排序算法，还是需要重视起来。能做到每一种排序算法都能理解原理，掌握算法流程以及时空复杂度，并且能用代码实现出来。

代码能力其实是慢慢培养的，大家写的每一行代码都是不会白费的。

## 2. 常见排序算法以及实现

> 后续所有排序算法，考虑的都是升序情况。只要能搞懂算法原理，逆序也是很容易实现的。
>
> 所有的排序算法的代码，都可以在第三部分的 **【模板】排序** 这道题中测试。

### 2.1 插入排序

**（可以用 ppt 演示流程）**

插入排序(Insertion Sort)类似于玩扑克牌插牌过程，每次将一个待排序的元素按照其关键字大小插入到前面已排好序的序列中，按照该种方式将所有元素全部插入完成即可。



**【代码实现】**

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

void insert_sort()
{
    // 依次枚举待排序的元素
    for(int i = 2; i <= n; i++) // 第一个位置默认就是有序的
    {
        int key = a[i];

        // 前面比 key 大的，统一右移
        int j = i - 1;
        while(j >= 1 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

```

```cpp
int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    insert_sort();

    for(int i = 1; i <= n; i++) cout << a[i] << " ";

    return 0;
}
```

**【时间复杂度】**

- 当整个序列有序的时候，插入排序最优，此时时间复杂度为 $O(n)$。
- 当整个序列逆序的时候，每个元素都要跑到最前面，时间复杂度为 $O(n^2)$。

## 2.2 选择排序

**（可以用 ppt 演示流程）**

选择排序(Selection Sort)是一种特别直观的排序算法。每次找出未排序序列中最小的元素，然后放进有序序列的后面。

**【代码实现】**

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

void selection_sort()
{
    for(int i = 1; i < n; i++) // 待排序区间的首位置
    {
        // [i, n] 区间就是待排序的区间
        int pos = i;
```

```
        for(int j = i + 1; j <= n; j++) // 查找待排序区间最小的元素的下标
        {
            if(a[j] < a[pos])
            {
                pos = j;
            }
        }

        swap(a[i], a[pos]);
    }
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    selection_sort();

    for(int i = 1; i <= n; i++) cout << a[i] << " ";

    return 0;
}
```

**【时间复杂度】**

- 无论数组有序还是无序，在未排序区间内找最小值的时候，都需要遍历一遍待排序的区间，因此时间复杂度为 $O(n^2)$。

## 2.3 冒泡排序

**（可以用 ppt 演示流程）**

冒泡排序(Bubble Sort)也是一种简单的排序算法。它的工作原理是每次检查相邻两个元素，如果前面的元素与后面的元素满足给定的排序条件，就将相邻两个元素交换。当没有相邻的元素需要交换时，排序就完成了。

由于在算法的执行过程中，较大的元素像是气泡般慢慢浮到数列的末端，故叫做冒泡排序。

**【代码实现】**

```
#include <iostream>
```

```cpp

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

// 优化后的冒泡排序
void bubble_sort()
{
    // 依次枚举待排序区间的最后一个元素
    for(int i = n; i > 1; i--)
    {
        bool flag = false;
        // [1, i] 就是待排序区间
        for(int j = 1; j < i; j++)
        {
            if(a[j] > a[j + 1])
            {
                swap(a[j], a[j + 1]);
                flag = true;
            }
        }

        if(flag == false) return;
    }
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    bubble_sort();

    for(int i = 1; i <= n; i++) cout << a[i] << " ";

    return 0;
}
```

**【时间复杂度】**

加上优化之后：

- 如果数组有序，只会扫描一遍，时间复杂度为 $O(n)$。
- 如果逆序，所有元素都会向后冒一次到合适位置，时间复杂度为 $O(n^2)$。

## 2.4 堆排序

**（可以用 ppt 演示流程）**

堆排序(Heap Sort)是指利用堆这种数据结构所设计的一种排序算法。本质上是优化了选择排序算法，如果将数据放在堆中，能够快速找到待排序元素中的最小值或最大值。

堆排序的过程分两步：

1. 建堆。升序建大堆，降序建小堆。

建堆过程，从倒数第一个非叶子节点开始，倒着一直到根结点位置，每个结点进行向下调整。

2. 排序。删除堆顶的逻辑。

每次将堆顶元素与堆中最后一个元素交换，然后将堆顶元素往下调整。直到堆中剩余一个元素，所有元素就都有序了。

因此，堆排序仅需用到向下调整算法。

**【代码实现】**

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

void down(int parent, int len)
{
    int child = parent * 2;

    while(child <= len)
    {
        if(child + 1 <= len && a[child + 1] > a[child]) child++;

        if(a[parent] >= a[child]) return;

```

```
            swap(a[parent], a[child]);
            parent = child;
            child = parent * 2;
        }
    }

    void heap_sort()
    {
        // 1. 建堆
        for(int i = n / 2; i >= 1; i--)
        {
            down(i, n);
        }

        // 2. 排序
        for(int i = n; i > 1; i--) // 枚举堆里面最后一个元素的位置
        {
            swap(a[1], a[i]);

            down(1, i - 1);
        }
    }

    int main()
    {
        cin >> n;
        for(int i = 1; i <= n; i++) cin >> a[i];

        heap_sort();

        for(int i = 1; i <= n; i++) cout << a[i] << " ";

        return 0;
    }
```

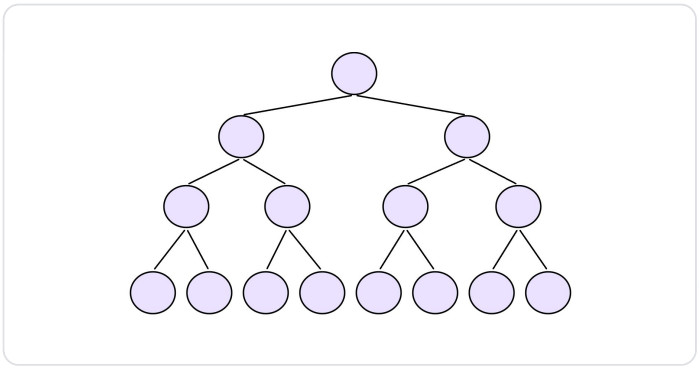
**【时间复杂度】**

时间复杂度需要计算两部分：一部分是建堆的时间复杂度，另一部分是排序。

- 排序部分的时间复杂度很好估计，每次都是从堆中选择一个最大值，然后与最后一个元素交换，然后调整。一次选择的时间复杂度为 $\log n$，一共执行 $n$ 次，大概就是 $n \log n$ 级别。
- 建堆的时间复杂度：

> 计算向下调整算法建堆时间复杂度

分析：



第1层，$2^0$ 个结点，需要向下移动h-1层

第2层，$2^1$ 个结点，需要向下移动h-2层

第3层，$2^2$ 个结点，需要向下移动h-3层

.....

第h-1层，$2^{h-2}$ 个结点，需要向下移动1层

则需要移动结点总的移动步数为：每层结点个数 * 向下调整次数

$$T(h) = 2^0 * (h-1) + 2^1 * (h-2) + 2^2 * (h-3) + 2^3 * (h-4) + .. + 2^{h-3} * 2 + 2^{h-2} * 1$$

这是一个等差数列乘以等比数列的形式，因此用错位相减法化简。

$$T(h) = 2^0 * (h-1) + 2^1 * (h-2) + 2^2 * (h-3) + 2^3 * (h-4) + .. + 2^{h-3} * 2 + 2^{h-2} * 1$$

$$2 * T(h) = 2^1 * (h-1) + 2^2 * (h-2) + 2^3 * (h-3) + 2^4 * (h-4) + ... + 2^{h-2} * 2 + 2^{h-1} * 1$$

错位相减：

$$T(h) = 1 - h + 2^1 + 2^2 + 2^3 + 2^4 + .. + 2^{h-2} + 2^{h-1}$$

$$T(h) = 2^0 + 2^1 + 2^2 + 2^3 + 2^4 + . + 2^{h-2} + 2^{h-1} - h$$

$$T(h) = 2^h - 1 - h$$

根据二叉树的性质：$n = 2^h - 1$ 和 $h = log_2(n+1)$

$$T(n) = n - log_2(n+1) \approx n$$

综上所述，堆排序的时间复杂度主要取决于排序的过程，也就是 $n \log n$。

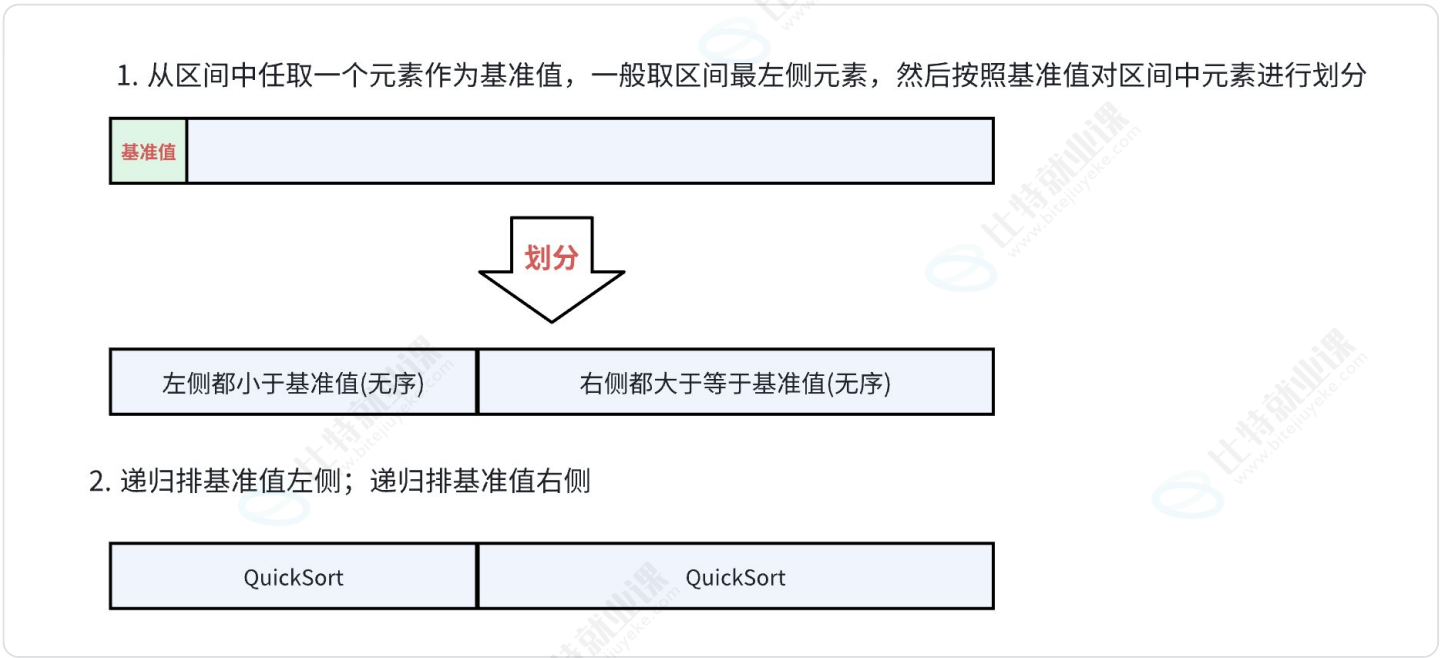
## 2.5 快速排序

**（可以用 ppt 演示流程）**

快速排序(Quick Sort)，既然敢起这样的名字，说明它是常见排序算法中较为优秀的。事实上，在很多情况下，快排确实是效率较高的算法。

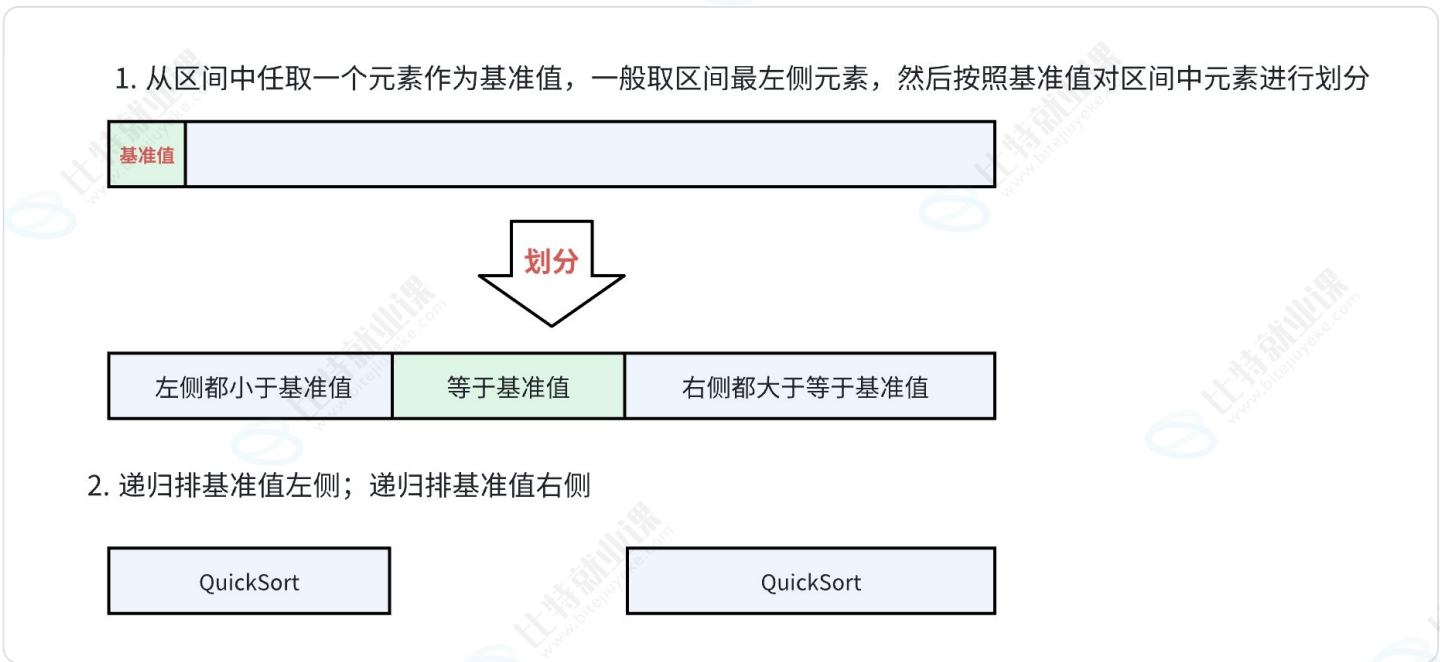
### 2.5.1 算法原理

常规快排：在待排序区间中任取一个元素作为枢轴(pivot，或称基准值，通常选取区间首元素)，然后按照基准值大小将区间中元素分割成左右两部分，左侧区间中元素小于基准值，右侧区间中元素大于等于基准值，即基准值已经放在其该放的位置上，该过程称为一次划分。划分结束后，再递归排基准值左侧，递归排基准值右侧即可。




## 2.5.2 优化一：三路划分

三路划分优化：其实可以做到按照基准元素，将所有元素分成三个区间。左部分全部小于 pivot，中间部分全部等于 pivot，右部分全部大于 pivot。然后中间部分就不用管了，直接递归处理左右部分。




而这个三路划分，就是典型的数组分三块算法，在顺序表章节讲过类似的算法题《颜色分类》，大家可以去回顾一下。

### 2.5.3 优化二：随机选择基准元素

选择基准元素的方式：

- 每次选择待排序序列的最左边元素

那么，当整个序列有序的时候，每次递归就会划分出特别长的一段右区间，递归的层数是 $n$ 次，每次要遍历整个数组一遍，时间复杂度就退化成 $n^2$ 。

每次选择最右边元素也是同理。

- 每次选择待排序序列的中间元素

可以构造一个特殊的序列，使得每次取中间元素的时候都会取到最小值，依旧会退化成 $n^2$ 。

- 随机选择基准元素

每次选择基准元素的时候，都从待排序的序列中随机选择一个数。在随机性的前提下，可以证明算法的时间复杂度趋近于 $n \log n$ 。（具体证明还需要用到概率论的知识，感兴趣的同学可以看看大黑书《算法导论》~）

因此，每次选择基准元素时，都使用随机函数选择。

**补充知识：C++ 中的随机函数。**

C++ 提供了函数 srand 和 rand，搭配使用可以返回一个随机值。

```cpp
#include <iostream>
#include <ctime>

using namespace std;

int main()
{
    srand(time(0)); // 种下一个随机数种子

    for(int i = 1; i <= 10; i++)
    {
        cout << rand() << endl; // 每次生成一个随机值
    }

    return 0;
}
```

**【代码实现】**

```cpp
#include <iostream>
#include <ctime>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

// 优化一：随机选择基准元素
int get_random(int left, int right)
{
    return a[rand() % (right - left + 1) + left];
}

void quick_sort(int left, int right)
{
    if(left >= right) return;

    // 1. 选择一个基准元素
    int p = get_random(left, right);

    // 2. 数组分三块 - 荷兰国旗问题
    int l = left - 1, i = left, r = right + 1;
    while(i < r)
    {
        if(a[i] < p) swap(a[++l], a[i++]);
        else if(a[i] == p) i++;
        else swap(a[--r], a[i]);
    }

    // [left, l] [l + 1, r - 1] [r, right]
    quick_sort(left, l);
    quick_sort(r, right);
}

int main()
{
    srand(time(0));
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    quick_sort(1, n);
```

```
45
46      for(int i = 1; i <= n; i++) cout << a[i] << " ";
47
48
49      return 0;
50  }
```

## 2.6 归并排序

**（可以用 ppt 演示流程）**

归并排序(Merge Sort)是无论数据有什么特性，时间复杂度就能稳定 $O(n \log n)$ 的排序算法。

归并排序用的是分治思想，不知道分治是啥也没关系，后续算法课会讲到。它的主要过程分两步：

1. 将整个区间从中间一分为二，先把左边和右边排排序；
2. 然后将左右两个有序的区间合并在一起。

其中，如何让左右两边有序，就继续交给归并排序，因此归并排序是用递归来实现的；合并两个有序区间的操作，在顺序表中讲过类似的算法题。

**【代码实现】**

```
1   #include <iostream>
2
3   using namespace std;
4
5   const int N = 1e5 + 10;
6
7   int n;
8   int a[N];
9   int tmp[N]; // 辅助归并排序时，合并两个有序数组
10
11  void merge_sort(int left, int right)
12  {
13      if(left >= right) return;
14
15      // 1. 先一分为二
16      int mid = (left + right) >> 1;
17      // [left, mid] [mid + 1, right]
18      // 2. 先让左右区间有序
19      merge_sort(left, mid);
20      merge_sort(mid + 1, right);
```

```cpp
    // 3. 合并两个有序数组
    int cur1 = left, cur2 = mid + 1, i = left;
    // [left, mid] [mid + 1, right]
    while(cur1 <= mid && cur2 <= right)
    {
        if(a[cur1] <= a[cur2]) tmp[i++] = a[cur1++];
        else tmp[i++] = a[cur2++];
    }

    while(cur1 <= mid) tmp[i++] = a[cur1++];
    while(cur2 <= right) tmp[i++] = a[cur2++];

    for(int j = left; j <= right; j++)
    {
        a[j] = tmp[j];
    }
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    merge_sort(1, n);

    for(int i = 1; i <= n; i++) cout << a[i] << " ";

    return 0;
}
```

**【时间复杂度】**

每次递归都是标准的一分为二，因此时间复杂度为 $O(n \log n)$。

## 3. 【模板】排序

> **题目来源：** 洛谷
>
> **题目链接：** P1177 【模板】排序
>
> **难度系数：** ★

**【题目描述】**

> 将读入的 $N$ 个数从小到大排序后输出。

**【输入描述】**

> 第一行为一个正整数 $N$。
>
> 第二行包含 $N$ 个空格隔开的正整数 $a_i$，为你需要进行排序的数。

**【输出描述】**

> 将给定的 $N$ 个数从小到大输出，数之间空格隔开，行末换行且无空格。

**【示例一】**

> 输入：
>
> ```
> 5
> 4 2 4 5 1
> ```
>
> 输出：
>
> ```
> 1 2 4 4 5
> ```

**【解法】**

看数据范围，需要使用堆排序或者快排或者归并排序。